

ASHLAND UNIVERSITY

Theory of Computation

via Inquiry-Based Learning

Iyad A. Ajwa

5/6/2011

A one semester undergraduate course in Theory of Computation focusing on the theory of finite state automata and context-free languages.

Contents

ACKNOWLEDGEMENTS.....	3
To The Instructor.....	4
Chapter 1.....	6
Mathematical Notations and Techniques.....	6
Section 1.1 Sets.....	6
Section 1.2 Relations.....	7
Section 1.3 Functions.....	8
Section 1.4 The “P” Word: Proofs.....	8
Section 1.5 Alphabets and Languages	9
Chapter 2.....	12
Finite State Automata	12
Section 2.1 Deterministic Finite State Automata	12
Section 2.2 Nondeterministic Finite State Automata	14
Section 2.3 Determinism vs. Nondeterminism	15
Section 2.4 Regular Languages	16
Section 2.5 Non-Regular Languages	18
Chapter 3.....	20
Context-Free Languages	20
Section 3.1 Context-Free Grammars.....	20
Section 3.2 Parsing.....	21
Section 3.3 Closure Properties.....	21
Section 3.4 Pushdown Automata.....	22
Section 3.5 Languages that are not Context-Free	24
REFERENCES.....	24

ACKNOWLEDGEMENTS

This guide was written with support from the Educational Advancement Foundation. The author gratefully acknowledges Harry Lucas and the Educational Advancement Foundation for their support.

In addition, the author gratefully acknowledges the extremely useful remarks of the first referee of the guide. The referee provided the author with insightful remarks and evaluations which have improved the author's understanding of the goals of inquiry-based learning.

Moreover, the author wishes to thank David Clark, Submissions Editor of the Journal of Inquiry-Based Learning in Mathematics, and the two anonymous reviewers of the transcript. Your comments and suggestions have been very helpful and are greatly appreciated.

To The Instructor

This guide has been developed for a one semester undergraduate course in Theory of Computation at Ashland University. Please allow me first to give you some background about our course.

Our course CS-302 (Theory of Computation) has been part of the curriculum since 1998. It is offered every fall semester and I have had the pleasure and privilege to teach it each time it has been offered. The course is required of all computer science majors and is elective for students majoring in mathematics. The course is an elective for students declaring computer science as their minor. Therefore, I always have a variety of students in the course.

The prerequisite for CS-302 is the Discrete Mathematics sequence, currently consisting of 2 courses. Our computer science majors are required to take the Discrete Mathematics sequence during their freshman year. On the other hand, our mathematics majors do not take the Discrete Mathematics sequence till their sophomore year. Therefore, on occasions, I have had sophomores in the class but the vast majority of students taking CS-302 have been at the junior and senior levels.

This guide has been used to teach CS-302 for twelve (12) times. Thus, the guide has gone through 12 revisions and refinements. The material has been presented to students in three formats: the traditional lecture format (the first 6 offerings of the course), a hybrid traditional/inquiry-based approach (the subsequent 4 offerings of the course), and inquiry-based setting (the last 2 offerings of the course). Still this is an ongoing project. However, I strongly believe that this guide, in its current form, can be beneficial to instructors who are interested in applying inquiry-based learning in teaching a similar course.

CS-302 focuses on the theory of finite state automata and context-free languages only. Thus, this guide has been organized into 3 chapters: Mathematical Notations and Techniques; Finite State Automata; and Context-Free Languages. Below are my remarks on the contents of each chapter.

Chapter 1 in this guide is basically a review of material covered in a Discrete Mathematics I course that is very relevant to Theory of Computation. This material includes sets, relations, functions, and mathematical proofs. It is assumed that students have thorough understanding of all concepts and proof techniques included in the chapter. However, you will be surprised how valuable this review is to students. I always spend the first 2 weeks of the semester reviewing these concepts and techniques and emphasizing where in the Theory of Computation course they will be encountered and used. Also included in this is the topic of alphabets and strings. Although this material could have been covered in the Discrete Mathematics sequence, my experience teaching this course has proven that this topic needs an additional 2 weeks to be fully understood by the students. It is at the core of the course and, therefore, needs to be fully comprehended. Currently, Chapter 1 consists of 34 definitions, 35 carefully selected exercises, and 10 supplementary items. It is possible that students have seen some of the exercises in their Discrete Mathematics course(s). The supplementary items are axioms, remarks, hints, notations, or other advice to students.

Chapter 2 presents the theory of finite state automata in detail. I usually spend good 4-5 weeks on this chapter. The material includes finite state automata (deterministic, nondeterministic, and their equivalence); regular expressions; regular languages and their relation to finite state automata; and the Pumping Lemma for regular languages. Currently, Chapter 2 consists of 21 definitions, 9 lemmas and theorems, 24 exercises, 2 construction algorithms, and 10 remarks. Here are a few comments on these items. Construction 33 (NFA to DFA) needs some attention. It is recommended that you as the instructor be involved in explaining the algorithm. It is unlikely that students be able to prove Theorem 35 on their own. It is recommended that you illustrate the theorem using an example before proving it with the students. Also note that Exercises 38 through 42 lead to the proof of Theorem 43. Lemma 48 and Construction 49 are used to prove Theorem 51.

Chapter 3 is on the theory of context-free languages. I usually spend 4-5 weeks on this chapter covering context-free grammars and their languages; derivation sequences; parse trees and the concept of ambiguous grammars; the closure theorem for context-free languages; pushdown automata; and the Pumping Lemma for context-free languages. Chapter 3 consists of 18 definitions, 3 lemmas and theorems, 19 exercises, and 2 remarks. Here are a few comments on these items. The proof that the language given in Exercise 24 is inherently ambiguous is very hard and beyond the scope of such an introductory course. However, the solution to the exercise has some nifty ideas. The exercise asks the student to provide an example of a context-free grammar for the language and then show that this grammar is ambiguous. As a hint, you may tell the students that they need to break the language L into two languages L_1 and L_2 such that $L = L_1 \cup L_2$. The students then can provide two context-free grammars for L_1 and L_2 . Finally, the students need to apply Exercise 17 to give a grammar for L . It is easy to show that the resulting grammar is ambiguous. Ask the students to think in terms of the strings that exist in $L_1 \cap L_2$. Those strings can be derived in two different ways via the grammars developed for L_1 and L_2 . The proof to Theorem 20 uses the construction proofs in Exercises 17, 18, and 19. The proof to Theorem 21 is beyond the scope of this course. But the theorem needs to be presented to students because it is needed for its applications. You may want to give the students a hint regarding Exercise 22: they need to express each of the two languages as the intersection of a context-free language and a regular language and then apply Theorem 21. Exercise 40 is proven easier using Theorem 21 rather than the Pumping Lemma (This is a proof by contradiction using Theorem 21 and the fact that $L_1 = L_3 \cap a^*b^*c^*$). The proof of Exercise 41 involves the languages in Exercises 38 and 40 together with Theorem 21. A useful hint to Exercise 42 is to note that $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ and connect that to Exercise 41 and Theorem 21.

Chapter 1

Mathematical Notations and Techniques

Section 1.1 Sets

1. **Definition.** A **set** is a collection of objects. The objects in a set are also called the *elements* or *members*, of the set.
2. **Notation.** If x is an element of a set A , then we write $x \in A$. Otherwise, we write $x \notin A$.
3. **Definition.** The **empty set** $\emptyset$ is the set with no elements. A **singleton set** is a set with exactly one element. The **Universal Set** U is the set of all objects under consideration.
4. **Definition.**
 - The union of two sets is: $A \cup B = \{x \in U \mid x \in A \text{ or } x \in B\}.$
 - The intersection of two sets is: $A \cap B = \{x \in U \mid x \in A \text{ and } x \in B\}.$
 - The difference between two sets is: $A - B = \{x \in U \mid x \in A \text{ and } x \notin B\}.$
 - The complement of a set is: $\bar{A} = \{x \in U \mid x \notin A\}.$
5. **Definition.** A set A is called a **subset** of another set B (denoted by $A \subseteq B$) if and only if every element of A is also an element of B . We see that $A \subseteq B$ if and only if the following quantification is true: $\forall x (x \in A \rightarrow x \in B)$.
6. **Exercise.** Prove:
 - a. Idempotent Laws: $A \cup A = A$ $A \cap A = A$
 - b. Absorption Laws: $A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$
 - c. DeMorgan's Laws: $A - (B \cup C) = (A - B) \cap (A - C)$ $A - (B \cap C) = (A - B) \cup (A - C)$
 - d. Commutative Laws: $A \cup B = B \cup A$ $A \cap B = B \cap A$
 - e. Associative Laws: $(A \cup B) \cup C = A \cup (B \cup C)$ $(A \cap B) \cap C = A \cap (B \cap C)$
 - f. Distributive Laws: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
7. **Exercise.** What is the simplest expression representing each of the following sets:
 - $(A - B) \cup (B - A) \cup (A \cap B)$
 - $A \cup (B \cap ((A - (B - A))))$
 - $\bar{A} \cup (B - (A \cup (\bar{B} - A)))$
8. **Definition.** Set A and set B are **equal** (denoted by $A = B$) if they consist of the same elements.
9. **Remark.** We note that $A = B$ if and only if $A \subseteq B$ and $B \subseteq A$. That is, $A = B$ if whenever $x \in A$, then $x \in B$ and whenever $x \in B$, then $x \in A$.
10. **Definition.** A set A is called a **proper subset** of another set B (denoted by $A \subset B$) if and only if $A \subseteq B$ and $A \neq B$.
11. **Definition.** The **power set** of a given set A (denoted by 2^A) is the set of all subsets (proper or not) of the set A .
12. **Exercise.** What is the power set of $\emptyset$? $\{\emptyset\}$?

- 13. Exercise.** What is the relationship between $2^{(A \cup B)}$ and $2^A \cup 2^B$? Under what circumstances are they equal? If they are not equal, is one necessarily a subset of the other, and if so, which one? Give reasons for your answers.
- 14. Exercise.** What is the relationship between $2^{(A \cap B)}$ and $2^A \cap 2^B$? Under what circumstances are they equal? If they are not equal, is one necessarily a subset of the other, and if so, which one? Give reasons for your answers.
- 15. Definition.** Two nonempty sets A and B are called **disjoint** if $A \cap B = \emptyset$.
- 16. Definition.** A collection $\mathcal{P}$ of nonempty subsets of A is said to be a **partition** of the set A if and only if the following three conditions are satisfied
1. each element of $\mathcal{P}$ is nonempty,
 2. distinct members of $\mathcal{P}$ are disjoint, and
 3. the union of members of $\mathcal{P}$ gives A .
- 17. Definition.** Let A be a set. If there are exactly n distinct elements in A where n is a nonnegative integer, we say that A is a **finite** set and that n is the **cardinality** of A . The cardinality of A is denoted by $|A|$. A set is said to be **infinite** if it is not finite.
- 18. Exercise.** Prove: If A is a finite set, then $|2^A| = 2^{|A|}$.

Section 1.2 Relations

- 19. Definition.** An **ordered pair** of elements, written (a, b) , is the *ordered* collection that has a as its first element and b as its second element.
- 20. Exercise.** Prove: $(a, b) = (c, d)$ if and only if $a = c \wedge b = d$.
- 21. Definition.** Let A and B be sets. The **Cartesian product** of A and B , denoted by $A \times B$, is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$. That is, $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$.
- 22. Exercise.** If $|A| = m$ and $|B| = n$, what is $|A \times B|$?
- 23. Definition.** A **binary relation** from a set A to set B is a subset of $A \times B$. A binary relation on a set A is a subset of $A \times A$.
- 24. Definition.** A relation R on a set A is called:
- **Reflexive** if $(a, a) \in R$ for each $a \in A$.
 - **Symmetric** if $(b, a) \in R$ whenever $(a, b) \in R$, for all $a, b \in A$.
 - **Transitive** if whenever $(a, b) \in R$ and $(b, c) \in R$, then $(a, c) \in R$, for all $a, b, c \in A$.
- 25. Exercise.** Let $\mathbb{N}$ be the set of natural numbers. Define a relation R on $\mathbb{N} \times \mathbb{N}$ by
- $$R = \{(x, y) \mid x \in \mathbb{N}, y \in \mathbb{N}, \text{ and } x + y \text{ is even}\}.$$
- Prove that R is reflexive, symmetric, and transitive.
- 26. Definition.** A relation R on a set A is called an **equivalence relation** if it is reflexive, symmetric, and transitive.

- 27. Definition.** Let R be an equivalence relation on a set A . For each $a \in A$, let $[a] = \{x \in A \mid (x, a) \in R\}$. The sets $[a]$ are called **equivalence classes** of the set A given by the relation R .
- 28. Exercise.** Let R be an equivalence relation on a set A . Let $P = \{[a] \mid a \in A\}$ be the set of all equivalence classes of the set A given by the relation R . Prove that P forms a partition of A .
- 29. Definition.** If a and b are integers and n is a positive integer, then **a is congruent to b modulo n** , denoted by $a \equiv_n b$, if $a - b$ is a multiple of n . That is, $a \equiv_n b$ if and only if $a - b = k \times n$ for some integer $k \geq 0$.
- 30. Exercise.** Prove that the congruence relation $\equiv_n$ is an equivalence relation for every integer $n > 0$.
- 31. Exercise.** Give the equivalence classes for the equivalence relation $\equiv_5$ defined on the natural numbers $\mathbb{N}$.

Section 1.3 Functions

- 32. Definition.** A **function** from a set A to a set B is a binary relation R on A and B with the following property: for each $a \in A$, there is exactly one ordered pair in R with first component a .
- 33. Exercise.** Is R of Exercise 25 a function?
- 34. Definition.** Let f be a function from a set A to a set B :
- We use the notation $f: A \rightarrow B$ to mean that f is a function from the set A to the set B .
 - If $(a, b) \in f$, we say that b is the **image of a** under function f and we write $f(a) = b$.
 - A is called the **domain** of the function f .
 - The **range** of the function f is the set of all images of elements of A .
- 35. Definition.** A function $f: A \rightarrow B$ is said to be
- **one-to-one** (or **injection**) if and only if $f(x) = f(y)$ implies that $x = y$ for all x and y in the domain of f .
 - **onto** (or **surjective**) if each element in B is the image under the function f of some element of A .
 - **bijection** if it is one-to-one and onto.
- 36. Exercise.** Let $A = \{a, b\}$ and $B = \{1, 2, 3\}$. Let f be the function $f: A \times B \rightarrow B$ defined as follows: $f(a, 1) = 2, f(a, 2) = 3, f(a, 3) = 1, f(b, 1) = 3, f(b, 2) = 3$, and $f(b, 3) = 1$. Prove or disprove that f is a bijection.

Section 1.4 The “P” Word: Proofs

- 37. Axiom.** A statement $P(n)$ is true for all natural numbers provided that the following are true:
- $P(0)$ is true
 - For all $n \geq 0$, if $P(n)$ is true, then $P(n + 1)$ is true.
- 38. Remark.** Axiom 37 is known as Mathematical induction. It is a method of mathematical proof typically used to prove that a given property P is true for all natural numbers n . A proof by mathematical induction typically consists of the following three steps:
- **Basis Step:** Prove P is true of 0.

- **Induction Hypothesis:** Assume P is true for some arbitrary positive number n .
- **Induction Step:** Prove P is true for $n + 1$.

39. Exercise. Use mathematical induction to prove that $n! \geq 2^{n-1}$.

40. Remark. Proof by contradiction is used as follows: Assume that the theorem is false and show that this assumption implies that some known fact is false, and hence the original assumption was false.

41. Definition. A number x is called rational if x can be written as $x = \frac{a}{b}$, for some integers a and b where $b \neq 0$ and a and b are relatively prime (that is, have no divisors in common).

42. Exercise. Use Definition 41 and Remark 40 to prove that $\sqrt{2}$ is irrational.

43. Remark. One way to prove that a statement is false is by exhibiting a counterexample.

44. Exercise. Use Remark 43 to disprove that $2^{A \cup B} \subseteq 2^A \cup 2^B$.

45. Axiom. If A and B are finite sets and $|A| > |B|$, then there is no one-to-one function from A to B .

46. Remark. The above axiom is only one simple version of The Pigeonhole Principle.

Section 1.5 Alphabets and Languages

47. Definition. An **alphabet** Σ is a finite set of symbols.

48. Definition. A **string** over an alphabet is a finite sequence of symbols from the alphabet. We will use u, v, x, y, z to denote strings.

49. Definition. The **empty string**, denoted by λ , is a string that has no symbols.

50. Definition. The set of all strings over an alphabet Σ is denoted by Σ^* .

51. Definition. The **length** of a string w , denoted by $|w|$ is its length as a sequence.

52. Exercise. What is $|\lambda|$?

53. Exercise. A string $w \in \Sigma^*$ can be thought of as a function $w: \{1, 2, \dots, |w|\} \rightarrow \Sigma$. What is $w(j)$ where $j = 1, 2, \dots, |w|$?

54. Definition. The **concatenation** of two strings x and y , denoted by xy , is the string x followed by the string y . Formally, $w = xy$ if and only if

- $|w| = |x| + |y|$
- $w(i) = x(i)$ for $i = 1, \dots, |x|$
- $w(|x| + j) = y(j)$ for $j = 1, \dots, |y|$

55. Exercise. Let w be a string of some alphabet Σ . What is $w\lambda$? What is λw ?

56. Exercise. Suppose Σ is an alphabet. It is obviously possible for two strings x and y over Σ to satisfy the condition $xy = yx$, since this condition is always satisfied if $y = \lambda$ or if $x = \lambda$. Is it possible under

the additional restriction that x and y are both nonempty? Either prove that this cannot occur, or describe precisely the circumstances under which it can.

57. Definition. For each string w and each natural number i , the string w^i is defined as:

$$w^i = \begin{cases} \lambda, & i = 0 \\ w^{i-1}w, & i > 0 \end{cases}$$

58. Definition. A string v is a **substring** of a string w if and only if there are strings x and y such that $w = xvy$. If $w = xv$ for some x , then v is a **suffix** of w . If $w = vy$ for some y , then v is a **prefix** of w .

59. Exercise. Show that λ is a substring, a prefix, and suffix of any string w .

60. Definition. The **reversal** of a string w , denoted by w^R , is defined as follows:

- If w is a string of length 0, then $w^R = w = \lambda$.
- If w is a string of length $n + 1$, then $w = ua$ (for some a in the alphabet and some string u of length n) and $w^R = au^R$.

61. Exercise. Prove that for any two strings w and x , $(wx)^R = x^Rw^R$.

62. Definition. A **language** L over an alphabet Σ is any set of strings over Σ . In other words, $L \subseteq \Sigma^*$.

63. Exercise. Let $\Sigma = \{0, 1\}$. The following are languages over Σ . Give strings that are in each of the following languages. Also, give strings that are not.

- $L_1 = \{\lambda, 0, 00, 001\}$
- $L_2 = \{w \in \Sigma^* : |w| \leq 8\}$
- $L_3 = \{w \in \Sigma^* : |w| \text{ is odd}\}$
- $L_4 = \{w \in \Sigma^* : |w|_0 = |w|_1\}$
- $L_5 = \{w \in \Sigma^* : |w| \geq 2 \text{ and } w \text{ begins and ends with } 1\}$
- $L_6 = \{w \in \Sigma^* : w = 0^k \text{ where } k = 0, 1, 2, 3, \dots\}$
- $L_7 = \{w \in \Sigma^* : w = 1^m \text{ where } m = 0, 1, 2, 3, \dots\}$
- $L_8 = \{w \in \Sigma^* : w = 0^n 1^n \text{ where } n = 0, 1, 2, 3, \dots\}$

64. Definition.

- The **union** of two languages is defined as: $L_1 \cup L_2 = \{w \in \Sigma^* : w \in L_1 \text{ or } w \in L_2\}$.
- The **intersection** of two languages is defined as: $L_1 \cap L_2 = \{w \in \Sigma^* : w \in L_1 \text{ and } w \in L_2\}$.
- The **complement** of a language is defined as: $\bar{L} = \{w \in \Sigma^* : w \notin L\}$.
- The **concatenation** of two languages is defined as:

$$L_1 L_2 = \{w \in \Sigma^* : w = w_1 w_2 \text{ where } w_1 \in L_1 \text{ and } w_2 \in L_2\}$$

65. Exercise. Let $\Sigma = \{0, 1\}$ and let L_9 and L_{10} be the following two languages over Σ . What is $L_9 L_{10}$?

$$L_9 = \{w \in \Sigma^* : w \text{ has an even number of } 0\text{'s}\}$$

$$L_{10} = \{w \in \Sigma^* : w \text{ starts with a } 0 \text{ and the rest are } 1\text{'s}\}$$

66. Exercise. Describe the language $L_{11} = L_6 L_7$.

67. Exercise. Describe the language $L_{12} = L_4 \cap L_{11}$.

68. Definition. For each language L and each natural number n , the language L^n is defined as:

$$L^n = \begin{cases} \lambda, & n = 0 \\ L^{n-1}L, & n > 0 \end{cases}$$

- 69. Remark.** The language L^k is the language L concatenated with itself k times with $L^0 = \{\lambda\}$.
- 70. Exercise.** Let $L = \{01, 1, 100\}$. What is L^2 ? What is L^3 ? What is the smallest k such that $110001110011 \in L^k$?
- 71. Definition.** The **Kleene Star** of a language L , denoted by L^* , is defined as $L^* = \bigcup_{i=0}^{\infty} L^i$. Moreover, $L^+ = \bigcup_{i=1}^{\infty} L^i$.
- 72. Remark.** Accordingly, $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$ and $L^+ = L^1 \cup L^2 \cup \dots$. Therefore, we can describe L^* to be the set of all strings that can be obtained by concatenating any number of strings of L .
- 73. Exercise.** Let $L = \{01, 1, 100\}$. Is $110001110011 \in L^*$?
- 74. Exercise.** Let L be a language. It is clear from Remark 72 that $\lambda \in L^*$. Must λ be in L^+ ? Under what circumstances do we get $\lambda \in L^+$?
- 75. Exercise.** Let L be a language. It is clear from Remark 72 that $L^+ \subseteq L^*$. Under what circumstances are they equal?
- 76. Exercise.** Let L_m and L_n be any languages. Prove: If $L_m \subseteq L_n$, then $L_m^* \subseteq L_n^*$.
- 77. Exercise.** Let $L_{14} = \{w \in \{0, 1\}^* : w \text{ has an unequal number of 0s and 1s}\}$. Show that $L_{14}^* = \{0, 1\}^*$.
- 78. Exercise.** Let L_m and L_n be finite languages. Is it always true that $|L_m L_n| = |L_m| \cdot |L_n|$? (For example, if L_m has 3 elements and L_n has 4, does the concatenation $L_m L_n$ always have 12?) Either prove it or find a counterexample.
- 79. Exercise.** In each case give an example of languages L_m and L_n that satisfy the condition $L_m L_n = L_n L_m$ and the given additional condition. Hint: Think of simple finite languages.
- Neither language is a subset of the other and neither language is $\{\lambda\}$.
 - L_m is a proper subset of L_n and $L_m \neq \{\lambda\}$.

Chapter 2

Finite State Automata

Section 2.1 Deterministic Finite State Automata

1. **Definition.** A **Deterministic Finite State Automaton (DFA)** is a quintuple $M = (Q, \Sigma, \delta, s, F)$ where:

Q is a finite set of objects we will call **states**
 Σ is a finite set of symbols we will call the **input alphabet**
 $\delta: Q \times \Sigma \rightarrow Q$ is called the **transition function**
 $s \in Q$ is called the **initial state**
 $F \subseteq Q$ is called the set of **final states**

2. **Definition.** A **configuration** C of a DFA is any element of $Q \times \Sigma^*$. A configuration of the form $C = (q, \lambda)$ indicates that the DFA has no input and hence its operation ceases at this point.
3. **Definition.** Let $C_1 = (q, w)$ and $C_2 = (p, u)$ be two configurations of a DFA. Then we say that C_1 **yields** C_2 **in one step** if and only if $w = \sigma u$ for some $\sigma \in \Sigma$, and $\delta(q, \sigma) = p$. If C_1 yields C_2 in one step, then we write $C_1 \vdash C_2$.
4. **Exercise.** Let $M = (Q, \Sigma, \delta, s, F)$ be the DFA where: $Q = \{p, t\}$, $\Sigma = \{0, 1\}$, $s = p$, $F = \{p\}$, and δ is the function tabulated below.

Current State q	Input Symbol σ	Next State $\delta(q, \sigma)$
p	0	p
p	1	t
t	0	t
t	1	p

Consider the configurations $C_1 = (p, 0110)$, $C_2 = (p, 110)$, $C_3 = (p, 10)$, $C_4 = (t, 10)$, $C_5 = (p, 0)$, and $C_6 = (p, \lambda)$. Which of the following statements is true? Why?

- $C_1 \vdash C_2$
- $C_2 \vdash C_4$
- $C_4 \vdash C_5$
- $C_2 \vdash C_3$
- $C_3 \vdash C_4$
- $C_5 \vdash C_6$

5. **Definition.** Let $C_i = (q, w)$ and $C_j = (p, u)$ be two configurations of a DFA. For $k \geq 1$, we write $C_i \vdash^k C_j$ if either $k = 1$ or $k > 1$ and there exists a configuration $C_m = (r, z)$ and $C_m \vdash^{k-1} C_j$. If $C_i \vdash^k C_j$ then we say that C_i **yields** C_j **in k steps**.
6. **Remark.** The definition above indicates that for C_i to yield C_j in k steps, we need to find a sequence of configurations $C_i, C_{i+1}, C_{i+2}, \dots, C_{j-2}, C_{j-1}, C_j$ such that each configuration in $C_i, C_{i+1}, \dots, C_{j-1}$ yields the next configuration in the sequence in one step. That is, $C_i \vdash C_{i+1} \vdash \dots \vdash C_{j-1} \vdash C_j$.
7. **Exercise.** Let $M = (Q, \Sigma, \delta, s, F)$ be the DFA in Exercise 4. In how many steps does C_1 yield C_4 ? Give the sequence of configurations.
8. **Definition.** Let $C_i = (q, w)$ and $C_j = (p, u)$ be two configurations of a DFA. We write $C_i \vdash^* C_j$ if either $(q, w) = (p, u)$ or $C_i \vdash^k C_j$ for some $k > 1$. If $C_i \vdash^* C_j$ then we say that C_i **yields** C_j **in zero or more steps**.

9. **Exercise.** Let $M = (Q, \Sigma, \delta, s, F)$ be the DFA in Exercise 4. Show that $C_1 \vdash^* C_6$.

10. **Definition.** A string $w \in \Sigma^*$ is said to be **accepted** by a DFA M if and only if there is a state $q \in F$ such that $(s, w) \vdash^* (q, \lambda)$ in M .

11. **Exercise.** Let $M = (Q, \Sigma, \delta, s, F)$ be the DFA in Exercise 4. Which of the following strings is accepted by M ?

- | | | |
|---------|-------------|----------------|
| • 00100 | • 0110 | • 011001110100 |
| • 01001 | • 111000101 | • 101010101011 |

12. **Definition.** The **language defined by** a DFA M , denoted by $L(M)$, is the set of all strings accepted by M . That is, $L(M) = \{w \in \Sigma^* : (s, w) \vdash^* (q, \lambda) \text{ in } M \text{ where } q \in F\}$.

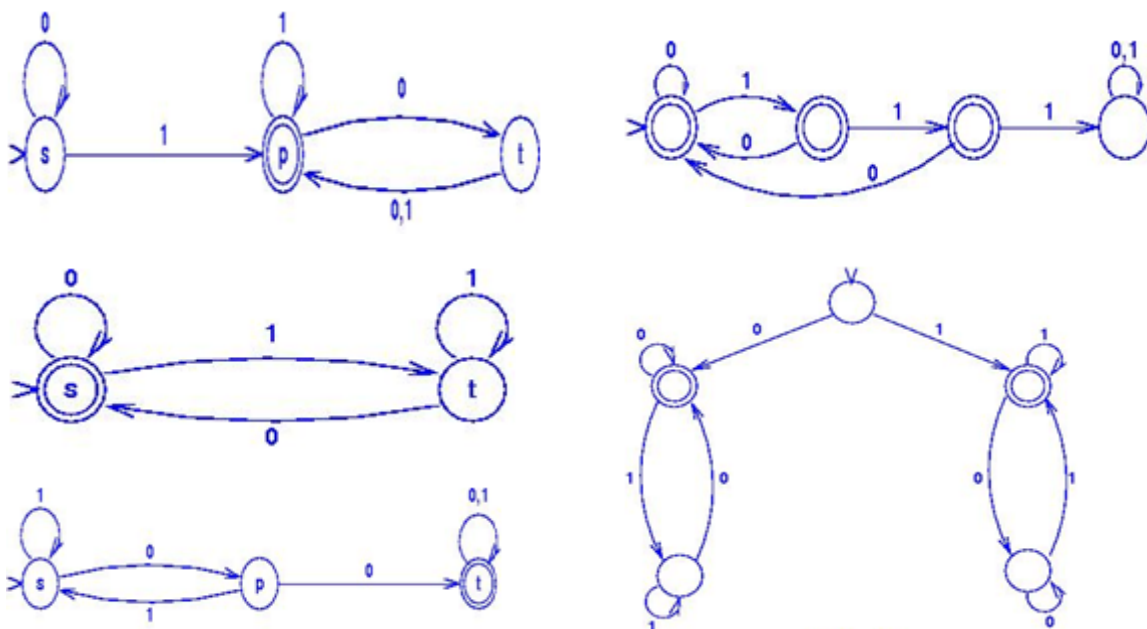
13. **Exercise.** Let $M = (Q, \Sigma, \delta, s, F)$ be the DFA in Exercise 4. What is the language of M ?

14. **Remark.** A convenient way to illustrate a DFA is to use its *state diagram*. The state diagram of a DFA is a directed graph, where nodes represent states and arrows are labeled with symbols from the input alphabet. The state diagram of a DFA can be constructed as follows. Examples of state diagrams follow in the following example.

- Each state in the DFA is represented as a node in the graph using a single circle.
- The initial state in the DFA is identified in the graph using the $\rightarrow$ symbol.
- The final states are identified in the graph as double circles.
- Each transition in the DFA $\delta(p, \sigma) = q$ is represented in the graph using an arrow from node p to node q and labeled with σ .

15. **Exercise.** Give the state diagram of the DFA M of Exercise 4.

16. **Exercise.** Give the language defined by each of the following five DFA.



Section 2.2 Nondeterministic Finite State Automata

17. Definition. A **Nondeterministic Finite State Automaton (NFA)** is a quintuple $M = (Q, \Sigma, \Delta, s, F)$

where:

Q is a finite set of objects we will call **states**

Σ is a finite set of symbols we will call **input alphabet**

$\Delta \subseteq Q \times (\Sigma \cup \{\lambda\}) \times Q$ is called the **transition relation**

$s \in Q$ is called the **initial state**

$F \subseteq Q$ is called the set of **final states**

18. Exercise. List two differences between a DFA and an NFA.

19. Definition. Each triple $(p, u, q) \in \Delta$ is called a **transition** of the NFA M . If the NFA M is in state p and the input symbol is σ , then M may next follow any transition of the form (p, σ, q) or (p, λ, q) .

20. Exercise. What happens (in terms of consuming input) if an NFA follows a transition (p, λ, q) ?

21. Definition. A **configuration** of an NFA is any element of $Q \times \Sigma^*$.

22. Definition. If $C_1 = (q, w)$ and $C_2 = (p, u)$ are two configurations of an NFA, then $C_1 \vdash C_2$ (**C_1 yields C_2 in one step**) if and only there is a $\sigma \in \Sigma \cup \{\lambda\}$ such that $w = \sigma u$ and $(q, \sigma, p) \in \Delta$.

23. Definition. Let $C_i = (q, w)$ and $C_j = (p, u)$ be two configurations of an NFA. For $k \geq 1$, we write $C_i \vdash^k C_j$ if either $k = 1$ or $k > 1$ and there exists a configuration $C_m = (r, z)$ and $C_m \vdash^{k-1} C_j$. If $C_i \vdash^k C_j$ then we say that **C_i yields C_j in k steps**.

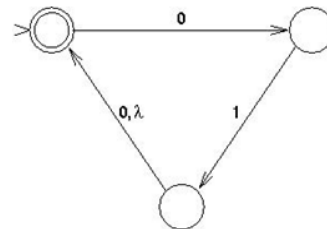
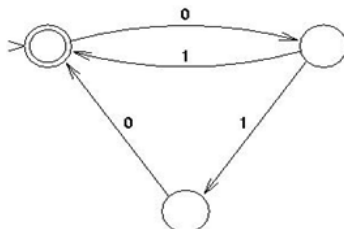
24. Remark. The definition above indicates that for C_i to yield C_j in k steps, we need to find a sequence of configurations $C_i, C_{i+1}, C_{i+2}, \dots, C_{j-2}, C_{j-1}, C_j$ such that each configuration in $C_i, C_{i+1}, \dots, C_{j-1}$ yields the next configuration in the sequence in one step. That is, $C_i \vdash C_{i+1} \vdash \dots \vdash C_{j-1} \vdash C_j$.

25. Definition. Let $C_i = (q, w)$ and $C_j = (p, u)$ be two configurations of a DFA. We write $C_i \vdash^* C_j$ if either $(q, w) = (p, u)$ or $C_i \vdash^k C_j$ for some $k > 1$. If $C_i \vdash^* C_j$ then we say that **C_i yields C_j in zero or more steps**.

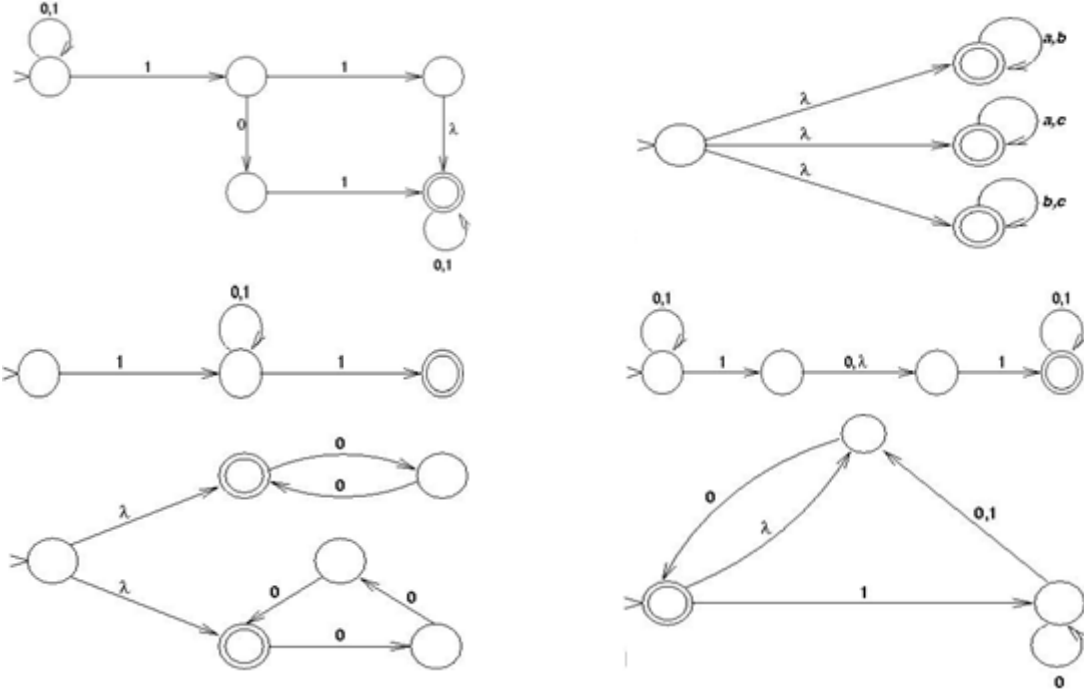
26. Definition. A string $w \in \Sigma^*$ is said to be **accepted** by an NFA M if and only if there is a state $q \in F$ such that $(s, w) \vdash^* (q, \lambda)$ in M .

27. Definition. The **language defined by** an NFA M , denoted by $L(M)$, is the set of all strings accepted by M . That is, $L(M) = \{w \in \Sigma^* : (s, w) \vdash^* (q, \lambda) \text{ in } M \text{ where } q \in F\}$.

28. Exercise. Give the language defined by each of the following NFA. What do you notice about the two languages?



29. Exercise. Give the language defined by each of the following NFA.



Section 2.3 Determinism vs. Nondeterminism

30. Definition. An NFA, $M = (Q, \Sigma, \Delta, s, F)$, is deterministic if and only if the following two conditions are true:

- There are no transitions of the form (p, λ, q) in Δ
- For each $q \in Q$ and $\sigma \in \Sigma$ there is exactly one $p \in Q$ such that $(q, \sigma, p) \in \Delta$

31. Definition. We say that two finite state automata (deterministic or nondeterministic) M_1 and M_2 are **equivalent** if and only if $L(M_1) = L(M_2)$

32. Exercise. Explain in simple words the equivalence of two finite state automata.

33. Construction (NFA to DFA).

1. Produce M_1 from M by eliminating all λ -transitions (if any) as follows:
 - a. If $(p, \lambda, q) \in \Delta$ and $(q, \lambda, r) \in \Delta$, then add the transition (p, λ, r) to Δ .
 - b. If $(p, \lambda, q) \in \Delta$ and $(q, \sigma, r) \in \Delta$, then add the transition (p, σ, r) to Δ .
 - c. If $(p, \lambda, f) \in \Delta$ and f is a final state, then make p a final state as well.
 - d. Eliminate all λ -transitions from the state diagram.
2. Produce M' from M_1 by applying the following Iterative Subset Construction Algorithm:

Let $Q_0 = \{s\}$ be the zeroth subset of Q and 0 be the corresponding state in Q'

Set $i = 0$ and set $last = 0$

While $i \leq last$ do

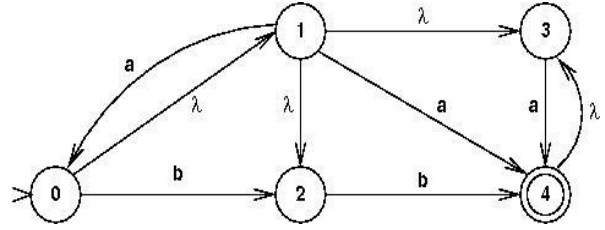
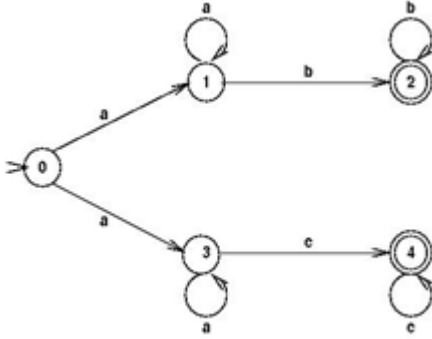
For each symbol of $\sigma \in \Sigma$ do

If $(Q_i, \sigma, Q_j) \notin \Delta$, for all $j, 0 \leq j \leq last$ then

$last = last + 1$

$\delta(i, \sigma) = \text{last}$
 Add $(Q_i, \sigma, Q_{\text{last}})$ to Δ
 $i = i + 1$
 Set $Q' = \{i : 0 \leq i \leq \text{last}\}$
 Set $F' = \{i : Q_i \cap F \neq \emptyset \text{ and } 0 \leq i \leq \text{last}\}$

34. Exercise. Find a DFA that is equivalent to the NFA shown below:



35. Theorem. Let $M = (Q, \Sigma, \Delta, s, F)$ be a nondeterministic finite state automaton. Define the deterministic finite state automaton $M' = (Q', \Sigma, \delta, s', F')$ where $Q' = 2^Q$, $s' = \{s\}$, $F' = \{R \in Q' \mid R \text{ contains a final state of } M\}$, and the transitions in δ are defined as follows: For $R \in Q'$, $\delta(R, \sigma) = \{q \in Q \mid (r, \sigma, q) \in \Delta \text{ for some } r \in R\}$. Then $L(M) = L(M')$

36. Remark. Observe that the DFA M' in Theorem 35 will have as many states as Q has subsets and that could cause a crucial problem, namely exponential increase in the number of states. The Iterative Subset Construction Algorithm ameliorates that by only including the states that can be *reached* from s' .

37. Theorem. For each nondeterministic finite state automaton, there exists an equivalent deterministic finite state automaton.

Section 2.4 Regular Languages

38. Exercise. Let $M_1 = (Q_1, \Sigma, \Delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \Delta_2, s_2, F_2)$ be nondeterministic finite state automata. Define the nondeterministic finite state automaton $M = (Q, \Sigma, \Delta, s, F)$ as follows:

$$\begin{aligned}
 Q &= Q_1 \cup Q_2 \cup \{s\} \\
 \Delta &= \Delta_1 \cup \Delta_2 \cup \{(s, \lambda, s_1), (s, \lambda, s_2)\} \\
 F &= F_1 \cup F_2
 \end{aligned}$$

Prove: $L(M) = L(M_1) \cup L(M_2)$.

39. Exercise. Let $M_1 = (Q_1, \Sigma, \Delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \Delta_2, s_2, F_2)$ be nondeterministic finite state automata. Define the nondeterministic finite state automaton $M = (Q, \Sigma, \Delta, s, F)$ as follows:

$$\begin{aligned}
 Q &= Q_1 \cup Q_2 \\
 \Delta &= \Delta_1 \cup \Delta_2 \cup \{(f, \lambda, s_2) \text{ for every } f \in F_1\} \\
 s &= s_1 \\
 F &= F_2
 \end{aligned}$$

Prove: $L(M) = L(M_1)L(M_2)$.

40. Exercise. Let $M_1 = (Q_1, \Sigma, \Delta_1, s_1, F_1)$ be a nondeterministic finite state automaton. Define the nondeterministic finite state automaton $M = (Q, \Sigma, \Delta, s, F)$ as follows:

$$\begin{aligned} Q &= Q_1 \cup \{s\} \\ \Delta &= \Delta_1 \cup \Delta_2 \cup \{(f, \lambda, s_1) \text{ for every } f \in F_1\} \cup \{(s, \lambda, s_1)\} \\ F &= F_1 \cup \{s\} \end{aligned}$$

Prove: $L(M) = L(M_1)^*$.

41. Exercise. Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ be a *deterministic* finite state automaton. Define the deterministic finite state automaton $M = (Q, \Sigma, \delta, s, F)$ as follows:

$$\begin{aligned} Q &= Q_1 \\ \delta &= \delta_1 \\ s &= s_1 \\ F &= Q_1 - F_1 \end{aligned}$$

Prove: $L(M) = \overline{L(M_1)}$.

42. Exercise. Show that if $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ are deterministic finite state automata, then we can construct a deterministic finite state automaton $M = (Q, \Sigma, \delta, s, F)$ such that $L(M) = L(M_1) \cap L(M_2)$. Hint: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$

43. Theorem. The class of languages accepted by a finite state automaton is closed under union, concatenation, Kleene Star, complementation, and intersection.

44. Definition. A **regular expression** is any string over the alphabet $\Sigma \cup \{\Phi, \Lambda, (,), +, *\}$ that can be obtained as follows:

- 1) $\alpha = \Phi$, $\alpha = \Lambda$, and $\alpha = \sigma$ each $\sigma \in \Sigma$ is a regular expression.
- 2) If α and β are regular expressions, then $\alpha\beta$ and $(\alpha\beta)$ are regular expressions.
- 3) If α and β are regular expressions, then $\alpha + \beta$ and $(\alpha + \beta)$ are regular expressions.
- 4) If α is a regular expression, then so is α^* .
- 5) Nothing else is a regular expression unless it follows from (1) through (4).

45. Remark. A regular expression describes a language exclusively by means of single symbols and Φ , combined perhaps with the symbols $+$ and $*$, possibly with the aid of parentheses. Thus, it is possible to map each regular expression α to a language $L(\alpha)$ as in the following definition.

46. Definition. The mapping between regular expressions and the languages represented by those regular expressions is defined as follows:

- $L(\Phi) = \emptyset$, $L(\Lambda) = \{\lambda\}$, and $L(\sigma) = \{\sigma\}$ for any $\sigma \in \Sigma$.
- If α and β are regular expressions, then
 - $L(\alpha\beta) = L(\alpha)L(\beta)$.
 - $L(\alpha + \beta) = L(\alpha) + L(\beta)$.
 - $L(\alpha^*) = (L(\alpha))^*$

47. Definition. A **regular language** is any language that is described by a regular expression.

48. Lemma. Every regular language is the language accepted by some finite state automaton. *Hint:* See Exercises 38-40.

49. Lemma. The language accepted by a finite state automaton is a regular language. *Hint:* See the following remark.

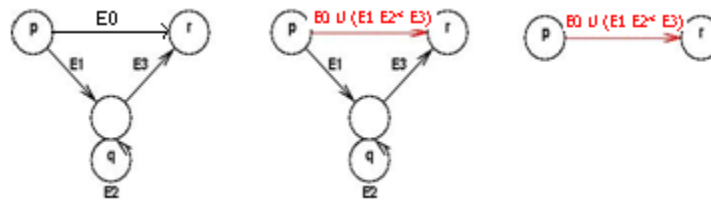
50. Remark. Given a deterministic finite state automaton $M = (Q, \Sigma, \delta, s, F)$, a regular expression that describes the language $L(M)$ can be constructed as follows:

1. Preparation Step.

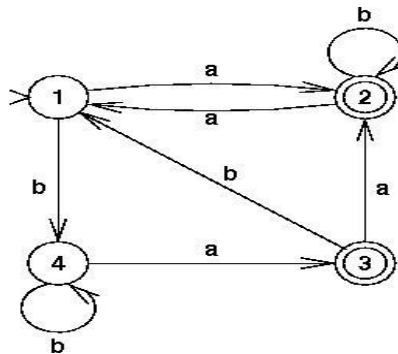
- It is required that no transitions go from any state into the start state. Therefore, if $\delta(q, s) \neq \emptyset$ for all q in Q , then add a new start state s' and the λ -transition (s', λ, s) .
- It is also required that there is only one final state. Therefore,
 - Add a new state, call it f , and make it final.
 - For each $q \in F$, add a λ -transition (q, λ, f) .
 - Make each q in F non-final.

2. State Elimination Step. Eliminate all states $q \in Q$ except for the final state f and the start state according to the following procedure:

- Whenever we have transitions (p, E_1, q) , possibly (q, E_2, q) , and (q, E_3, r) ; we add the transition $(p, E_1 E_2^* E_3, r)$.



51. Exercise. Given the following DFA, find a regular expression that describes the language of the DFA.



52. Theorem. A language is regular if and only if it is accepted by a finite state automaton.

Section 2.5 Non-Regular Languages

53. The Regular Languages Pumping Lemma. Let L be a regular language. There exists an integer $n > 0$ such that any string $w \in L$ with length $|w| \geq n$ can be written as $w = xyz$ such that,

- 1) $y \neq \lambda$,
- 2) $|xy| \leq n$, and
- 3) $xy^i z \in L$ for each $i \geq 0$.

54. Exercise. Prove that each of the following languages is not regular:

- $L_1 = \{a^k b^k : k \geq 0\}$

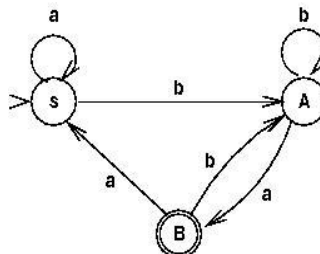
- $L_2 = \{a^k : k \text{ is prime}\}$
- $L_3 = \{w \in \{a, b\}^* : |w|_a = |w|_b\}$
- $L_4 = \{a^{k^2} : k \geq 0\}$
- $L_5 = \{w \in \{a, b\}^* : w = w^R\}$. *Hint:* Use Theorem 49 and the fact that L_1 is not regular.

Chapter 3

Context-Free Languages

Section 3.1 Context-Free Grammars

1. **Definition.** A **context-free grammar** (CFG) is a quadruple $G = (V, \Sigma, R, S)$, where
 V is a set of symbols we will call **an alphabet**
 $\Sigma \subseteq V$ is a set of input symbols that we will call **terminals**
 $R \subseteq (V - \Sigma) \times V^*$ is a set of **rules**
 $S \in (V - \Sigma)$ is the **start symbol**
2. **Definition.** Members of $V - \Sigma$ are called **nonterminals**.
3. **Definition.** For any $A \in (V - \Sigma)$ and $u \in V^*$, we write $A \rightarrow_G u$ whenever $(A, u) \in R$.
4. **Definition.** We write $u \Rightarrow_G v$ if and only if there exist strings $x, y \in V^*$ and $A \in V - \Sigma$ such that $u = xAy$, $v = xzy$, and $A \rightarrow_G z$. If $u \Rightarrow_G v$, we say that **u derives v in one step**.
5. **Definition.** We call any sequence of the form $w_0 \Rightarrow w_1 \Rightarrow \dots \Rightarrow w_n$ a **derivation** of w_n from w_0 . We also say that the derivation has n steps and we write $w_0 \Rightarrow^n w_n$. We write $w_0 \Rightarrow^* w_n$ if either $w_0 = w_n$ or $w_0 \Rightarrow^n w_n$ for some $n > 1$. If $w_0 \Rightarrow^* w_n$ then we say that **w_0 derives w_n in zero or more steps**.
6. **Definition.** The **language generated** by a CFG G , denoted by $L(G)$, is $L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}$.
7. **Definition.** A language L is said to be a **context-free language** (CFL) if and only if $L = L(G)$ for some context-free grammar G .
8. **Exercise.** Give the language generated by each of the following grammars:
 - $G_1 = (V, \Sigma, R, S)$, where $V = \{S, a, b\}$, $\Sigma = \{a, b\}$, and $R = \{S \rightarrow aSb \mid \lambda\}$.
 - $G_2 = (V, \Sigma, R, S)$, where $V = \{S, a, b\}$, $\Sigma = \{a, b\}$, and $R = \{S \rightarrow aS \mid Sb \mid \lambda\}$.
 - $G_3 = (V, \Sigma, R, S)$, where $V = \{S, a, b\}$, $\Sigma = \{a, b\}$, and $R = \{S \rightarrow aSa \mid bSb \mid \lambda\}$.
 - $G_4 = (V, \Sigma, R, S)$, where $V = \{S, (,)\}$, $\Sigma = \{(,)\}$, and $R = \{S \rightarrow (S) \mid SS \mid \lambda\}$.
9. **Exercise.** Consider the language accepted by the DFA $M = (Q, \Sigma, \delta, s, F)$. The same language is generated by the CFG $G = (V, \Sigma, R, S)$, where $V = Q \cup \Sigma$, $S = s$, and $R = \{p \rightarrow \sigma q : \delta(p, \sigma) = q\} \cup \{f \rightarrow \lambda : f \in F\}$. What can we say about regular languages and context-free languages?
10. **Exercise.** Construct a CFG that generates the language accepted by the following DFA.



Section 3.2 Parsing

- 11. Remark.** Derivation is a nondeterministic process. Any string can potentially have many different derivations. We can choose to always replace the leftmost nonterminal. Such derivations are called **leftmost derivations**. We can always replace the rightmost nonterminal. Such derivations are called **rightmost derivations**.
- 12. Definition.** We can picture the derivation process using a tree structure. Such a tree is called a **parse tree**. Each node in a parse tree is labeled by a nonterminal, say, A (with nonterminal S assigned to the root node). A is then connected to its children that are labeled $A_1, A_2, \dots, A_n$, where $A \rightarrow A_1 A_2 \dots A_n$ is a rule in the grammar. All leaves are labeled with terminals or λ . The concatenation of all leaves forms the string obtained by any derivation represented by the given parse tree.
- 13. Definition.** Grammars with strings having two or more distinct parse trees are called **ambiguous**.
- 14. Exercise.** Show that the grammar $G = (V, \Sigma, R, S)$, where $V = \{S, A, B, a\}$, $\Sigma = \{a\}$, and R is the following set of rules; is ambiguous.
- $$\begin{aligned} S &\rightarrow A \mid B \mid AB \\ A &\rightarrow aA \mid \lambda \\ B &\rightarrow aB \mid \lambda \end{aligned}$$
- 15. Remark.** The “Dangling ELSE Problem” is a well-known problem in computer programming. It is a situation in which it is not clear which part of a compound conditional statement an ELSE instruction belongs. It is basically a problem in some programming languages where an ELSE doesn't match the IF it is intended to match.
- 16. Exercise.** The Dangling ELSE Problem is a well-known problem in computer programming in which a seemingly well-defined grammar can become ambiguous. We often cite the Dangling ELSE Problem as a good example of how ambiguity relates to compiling in computer science. Restate the Dangling ELSE Problem in terms of ambiguity.
- 17. Exercise.** Show that the grammar $G = (V, \Sigma, R, S)$, where $V = \{S, A, B, a, +, *, (,)\}$, $\Sigma = \{a, +, *, (,)\}$, and R is the following set of rules; is not ambiguous.
- $$\begin{aligned} S &\rightarrow S + A \mid A \\ A &\rightarrow A * B \mid B \\ B &\rightarrow a \mid (S) \end{aligned}$$

Section 3.3 Closure Properties

- 18. Exercise.** Let $G_1 = (V_1, \Sigma_1, R_1, S_1)$ and $G_2 = (V_2, \Sigma_2, R_2, S_2)$ be two context-free grammars, and without loss of generality assume that they have disjoint sets of nonterminals. Let S be a new symbol and let $G = (V, \Sigma, R, S)$ be the context-free grammar where
- $$\begin{aligned} V &= V_1 \cup V_2 \cup \{S\} \\ \Sigma &= \Sigma_1 \cup \Sigma_2 \\ R &= R_1 \cup R_2 \cup \{S \rightarrow S_1, S \rightarrow S_2\} \end{aligned}$$
- Show that $L(G) = L(G_1) \cup L(G_2)$.
- 19. Exercise.** Let $G_1 = (V_1, \Sigma_1, R_1, S_1)$ and $G_2 = (V_2, \Sigma_2, R_2, S_2)$ be two context-free grammars, and without loss of generality assume that they have disjoint sets of nonterminals. Let S be a new symbol and let $G = (V, \Sigma, R, S)$ be the context-free grammar where

$V = V_1 \cup V_2 \cup \{S\}$
 $\Sigma = \Sigma_1 \cup \Sigma_2$
 $R = R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}$
 Show that $L(G) = L(G_1)L(G_2)$.

- 20. Exercise.** Let $G_1 = (V_1, \Sigma_1, R_1, S_1)$ be a context-free grammar. Let S be a new symbol and let $G = (V, \Sigma, R, S)$ be the context-free grammar where
- $V = V_1 \cup \{S\}$
 $\Sigma = \Sigma_1$
 $R = R_1 \cup \{S \rightarrow SS_1, S \rightarrow \lambda\}$
 Show that $L(G) = L(G_1)^*$

21. Theorem. The context-free languages are closed under union, concatenation, and the Kleene Star operation.

22. Theorem. The intersection of a context-free language with a regular language is a context-free language.

23. Exercise. Show that the following two languages are context-free:

$$L_1 = \{w \in \{a, b\}^* : |w|_a = |w|_b \text{ but containing no substring } abaa \text{ or } babb\}$$

$$L_2 = \{a^n b^n : n \geq 0 \text{ except for the string } aaabbb\}$$

Section 3.4 Pushdown Automata

24. Remark. Finite state automata discussed in Chapter 2 fail to recognize a simple CFL such as $L = \{wcw^R : w \in \{a, b\}^*\}$. The reason is that finite state automata do not have the capability to “memorize” any part of input processed. That is, there is no capability to “store” symbols. A device that would enable a finite state automaton to store data is known as the *stack*. It is a device that allows a finite state automaton to store data in the form of a list of symbols. However, access to this data is limited to only one end of the list, known as the *top* of the stack. So, if a symbol is to be added to the stack, it will be placed on top of the stack; and if a symbol is removed from the stack, only the symbol on top of the stack can be removed.

25. Definition. A **pushdown Automaton** is a sextuple $M = (Q, \Sigma, \Gamma, \Delta, s, F)$, where:

- Q is a finite set of **states**
- Σ is an alphabet (the **input symbols**)
- Γ is an alphabet (the **stack symbols**)
- Δ is the **transition relation**, $\Delta \subseteq (Q \times (\Sigma \cup \{\lambda\}) \times \Gamma) \times (Q \times \Gamma^*)$
- s is the **initial state**, $s \in Q$
- F is the set of **final states**, $F \subseteq Q$

26. Definition. A **transition** of a pushdown automaton M is a pair $((p, \sigma, \beta), (q, \gamma)) \in \Delta$. If $((p, \sigma, \beta), (q, \gamma)) \in \Delta$ is a transition of a pushdown automaton M , then whenever M is in state p with β at the top of the stack, M may read σ from the input tape, replace β by γ on the top of the stack, and enter state q . If $\sigma = \lambda$, then the input is not consulted.

27. Definition. We use the **Push** operation to add a symbol to the top of the stack. The transition used to push σ is $((p, u, \lambda), (q, \sigma))$. Similarly, we use the **Pop** operation to remove a symbol from the top of the stack. The transition used to pop σ is $((p, u, \sigma), (q, \lambda))$.

28. Definition. A **configuration** of a pushdown automaton M is defined to be a member of $Q \times \Sigma^* \times \Gamma^*$.

29. Definition. If $C_1 = (p, x, \alpha)$ and $C_2 = (q, y, \zeta)$ are configurations of M , we say that C_1 **yields** C_2 **in one step** (written as $C_1 \vdash_M C_2$) if there is a transition $((p, \sigma, \beta), (q, \gamma)) \in \Delta$ such that $x = \sigma y$, $\alpha = \beta \eta$, and $\zeta = \gamma \eta$ for some $\eta \in \Gamma^*$.

30. Definition. Any sequence of configurations $C_0, C_1, \dots, C_n$ such that $C_i \vdash_M C_{i+1}$ for $i = 0, 1, \dots, n$ will be called a **computation** by M and we write $C_0 \vdash^n C_n$. We write $C_0 \vdash^* C_n$ if either $C_0 = C_n$ or $C_0 \vdash^n C_n$ for some $n > 1$.

31. Definition. We say that a pushdown automaton M **accepts** a string $w \in \Sigma^*$ if and only if $(s, w, \lambda) \vdash^* (f, \lambda, \lambda)$ for some $f \in F$.

32. Definition. The **language accepted** by a pushdown automaton M , denoted $L(M)$, is the set of all strings accepted by M . That is, $L(M) = \{w \in \Sigma^* \mid (s, w, \lambda) \vdash^* (f, \lambda, \lambda) \text{ for some } f \in F\}$.

33. Exercise. Let $M_1 = (Q, \Sigma, \Gamma, \Delta, s, F)$ be the pushdown automaton where $Q = \{s, p, f\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a\}$, $F = \{s, f\}$, and Δ consists of the following transitions:

$((s, a, \lambda), (p, a))$
 $((p, a, \lambda), (p, a))$
 $((p, b, a), (f, \lambda))$
 $((f, b, a), (f, \lambda))$

What is $L(M_1)$?

34. Exercise. Let $M_2 = (Q, \Sigma, \Gamma, \Delta, s, F)$ be the pushdown automaton where $Q = \{s, f\}$, $\Sigma = \{a, b, c\}$, $\Gamma = \{a, b\}$, $F = \{f\}$, and Δ consists of the following transitions:

$((s, a, \lambda), (s, a))$
 $((s, b, \lambda), (s, b))$
 $((s, c, \lambda), (f, \lambda))$
 $((f, a, a), (f, \lambda))$
 $((f, b, b), (f, \lambda))$

What is $L(M_2)$?

35. Exercise. Construct a pushdown automaton $M_3 = (Q, \Sigma, \Gamma, \Delta, s, F)$ that accepts the language $\{ww^R : w \in \{a, b\}^*\}$.

36. Exercise. Let $M_4 = (Q, \Sigma, \Gamma, \Delta, s, F)$ be the pushdown automaton where $Q = \{s, p, f\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b, c\}$, $F = \{f\}$, and Δ consists of the following transitions:

$((s, \lambda, \lambda), (p, c))$
 $((p, a, c), (p, ac))$
 $((p, a, a), (p, aa))$
 $((p, a, b), (p, \lambda))$
 $((p, b, c), (p, bc))$
 $((p, b, b), (p, bb))$
 $((p, b, a), (p, \lambda))$
 $((p, \lambda, c), (f, \lambda))$

What is $L(M_4)$?

Section 3.5 Languages that are not Context-Free

37. CFL Pumping Lemma. Let $G = (V, \Sigma, R, S)$ be a context-free grammar. Then there exists an integer $n > 0$ such that any string $w \in L(G)$ with length $|w| \geq n$ can be written as $w = uvxyz$ for some strings $u, v, x, y, z \in \Sigma^*$ such that

- 1) $|v| > 0$ or $|y| > 0$
- 2) $|vxy| \leq n$
- 3) $uv^i xy^i z \in L(G)$ for any $i \geq 0$

38. Exercise. Prove that the language $L_1 = \{a^k b^k c^k : k \geq 0\}$ is not context-free.

39. Exercise. Prove that the language $L_2 = \{a^p : p \text{ is a prime}\}$ is not context-free.

40. Exercise. Prove that the language $L_3 = \{w \in \{a, b, c\}^* : |w|_a = |w|_b = |w|_c\}$ is not context-free.

41. Exercise. Prove or disprove: The class of context-free languages is closed under intersection.

42. Exercise. Prove or disprove: The class of context-free languages is closed under complementation.

REFERENCES

1. Efim Kinber and Carl Smith, Theory of Computation: A Gentle Introduction, Prentice Hall, ISBN: 0-13-027961-7, 2001.
2. Harry R. Lewis and Christos H. Papadimitriou, Elements of the Theory of Computation, Prentice Hall, Second Edition, ISBN: 0-13-262478-8, 1998.
3. John C. Martin, Introduction to Languages and the Theory of Computation, Second Edition, McGraw-Hill, ISBN: 0-07-040845-9, 1997.
4. Michael Sipser, Introduction to the Theory of Computation, PWS Publishing Company, ISBN: 0-534-94728-X, 1997.
5. Derrick Wood, Theory of Computation, Harper & Row, ISBN: 0-06-047208-1, 1987.